Empirical Analysis of Post-Quantum Cryptographic Handshake Resilience in Lossy IoT Networks: A Comparative Study of Kyber-TLS and ECC-TLS Under Adverse Network Conditions

Maisam Raza
2023463
Ghulam Ishaq Khan Institute

Abdullah Ahtasham
2023033
Ghulam Ishaq Khan Institute

Muhammad Shaheem
2023507
Ghulam Ishaq Khan Institute

***Abstract*—This paper presents an empirical evaluation of Post-Quantum Cryptographic (PQC) handshake resilience in constrained IoT environments, comparing Kyber-512-based TLS (KYBER-TLS) against classical Elliptic Curve Cryptography TLS (ECC-TLS). Extending the PQC-IoTNet benchmarking architecture, we conducted 1,260 controlled trials across seven packet loss levels (0–10%) and three jitter conditions (0, 20, 50 ms) using Linux NetEm on an MQTT/TLS protocol stack. Both algorithms achieved 100% connection success across all tested conditions. However, KYBER-TLS demonstrated a 32.4% lower mean handshake latency at baseline (397.9 ms vs. 588.8 ms) and consistently lower variance under adverse conditions. The Jitter Sensitivity Index remained near 1.0 for both algorithms, indicating comparable jitter resilience. These results provide statistically rigorous evidence that KYBER-TLS is a viable, performance-superior replacement for ECC-TLS in lossy IoT deployments, supporting post-quantum migration readiness.**

***Index Terms*—Post-Quantum Cryptography, Kyber, TLS, IoT Security, MQTT, Network Emulation, Handshake Latency, ECC, NetEm, Jitter Sensitivity.**

I. INTRODUCTION

The global deployment of Internet of Things (IoT) devices has accelerated dramatically, with projections exceeding 29 billion connected devices by 2030 [1]. These devices frequently operate over wireless channels—5G, Wi-Fi 6, LoRaWAN—where packet loss and latency variance are endemic rather than exceptional. Securing the communication layer of such devices demands cryptographic protocols that are not only quantum-resistant but also operationally robust under real-world network impairments.

The impending threat posed by Shor's algorithm to Elliptic Curve Cryptography (ECC) and RSA has prompted NIST to standardise a new family of Post-Quantum Cryptographic (PQC) algorithms [2]. Among them, CRYSTALS-Kyber (Kyber)—a lattice-based Key Encapsulation Mechanism (KEM)—was selected as the primary PQC standard for key establishment. While Kyber's theoretical advantages are well documented, its *practical* performance under adverse network conditions in resource-constrained IoT deployments remains insufficiently characterised.

Almutairi and Sheldon [3] proposed the PQC-IoTNet framework for benchmarking PQC handshakes in IoT contexts. However, their evaluation was limited to only 0% and 5% packet loss levels, 20 trials per scenario, and no jitter testing—leaving critical research gaps around threshold behaviour and statistical confidence. This work directly addresses these gaps.

Our contributions are:

1) A high-resolution packet loss gradient (seven levels from 0–10%) with 30 trials per condition;
2) Systematic jitter injection at 0, 20, and 50 ms across all conditions;
3) A comprehensive statistical analysis including mean, median, standard deviation, 95th percentile, and worst-case latency;
4) A Jitter Sensitivity Index (JSI) characterising relative susceptibility of each algorithm to latency variance; and
5) A complete dataset of 1,260 trials for future PQC benchmarking research.

II. BACKGROUND AND RELATED WORK

A. Post-Quantum Cryptography and Kyber

CRYSTALS-Kyber is a lattice-based KEM founded on the Module Learning With Errors (MLWE) problem [4]. Kyber-512 targets NIST security Level 1 (equivalent to AES-128) and produces compact key sizes: a public key of 800 bytes and a ciphertext of 768 bytes. These are significantly smaller than many other PQC candidates, making Kyber particularly attractive for constrained environments. The Open Quantum Safe (OQS) project provides production-ready integrations of Kyber into OpenSSL through the `oqs-provider` plugin [5], enabling transparent substitution of the ECC key exchange within standard TLS 1.3 handshakes.

B. TLS Handshake Performance in IoT

TLS 1.3 reduced the handshake to a single round-trip, significantly lowering overhead compared to TLS 1.2. Nevertheless, IoT networks commonly experience packet loss rates of 1–10% and jitter of tens of milliseconds —conditions under which retransmission timeouts can cause non-linear latency

TABLE I
PROTOCOL STACK USED IN ALL EXPERIMENTS

Layer	Component
Application	MQTT v3.1.1 (Eclipse Paho Python client)
Broker	Mosquitto 2.0 with TLS on port 8883
Security	TLS 1.3: ECC-P256 *or* Kyber-512 (OQS-provider)
Transport	TCP over loopback interface (`lo`)
Emulation	Linux NetEm (`tc qdisc`) for loss / delay / jitter

TABLE II
FULL EXPERIMENTAL DESIGN MATRIX

Variable	Values	Rationale
Packet Loss	0, 1, 2, 3, 5, 7, 10 %	High-res. gradient vs. 2 pts in [3]
Jitter	0, 20, 50 ms	Stable, moderate, high-variance links
Base Delay	50 ms (fixed)	WAN/satellite latency floor
Trials/condition	30	Reliable σ estimates
Algorithms	ECC-TLS, Kyber-TLS	Baseline vs. PQC candidate
Total trials	1,260 (630 per algo.)	15× the trial count of [3]

spikes [6]. Prior work by Pöppelmann et al. [7] demonstrated that post-quantum KEMs introduce larger message sizes, potentially amplifying retransmission probability under lossy links. Evaluating this amplification empirically remains an open problem.

C. The PQC-IoTNet Framework

Almutairi and Sheldon [3] introduced PQC-IoTNet, a benchmarking architecture that embeds Kyber into an MQTT/TLS stack using Mosquitto and OQS-OpenSSL. Their results showed KYBER-TLS performing comparably to ECC-TLS, but their methodology was limited in granularity and statistical power. Specifically, only two packet loss levels were tested, the trial count of 20 is insufficient for reliable standard deviation estimates, and jitter—a primary characteristic of 5G NR and satellite IoT links—was not evaluated. Our work extends PQC-IoTNet to close these gaps.

III. METHODOLOGY

A. Experimental Architecture

All experiments were conducted on an Ubuntu 22.04 LTS host (WSL2, Intel Core i7, 16 GB RAM). The protocol stack is summarised in Table I.

B. Experimental Design

The full experimental matrix crossed two algorithms with seven packet loss levels and three jitter levels, yielding 42 unique conditions at 30 trials each (Table II).

TABLE III
BASELINE STATISTICS (0% LOSS, 0 MS JITTER, $n = 30$ EACH)

Metric	ECC-TLS	KYBER-TLS	Δ
Mean (ms)	588.8	397.9	**−32.4%**
Median (ms)	589.9	397.3	**−32.7%**
Std Dev (ms)	48.4	62.3	¬+28.7%
Min (ms)	522.3	319.4	**−38.8%**
Max (ms)	727.1	547.2	**−24.7%**
P95 (ms)	678.4	497.6	**−26.7%**

C. Measurement Procedure

Each trial measured the *TLS handshake time*: the elapsed wall-clock time from the client's `connect()` call until TCP-level connection was confirmed with mutual TLS authentication complete. Python's `time.perf_counter()` provided sub-millisecond timing resolution. Network conditions were applied via:

```
tc qdisc add dev lo root netem delay Dms Jms loss
```

Conditions were cleared and re-applied between every scenario block to prevent state accumulation. ECC-TLS used a P-256 certificate chain with a CA-signed server certificate. KYBER-TLS used the same CA infrastructure with the `p256_kyber512` hybrid group via OQS-provider to maintain certificate compatibility.

D. Metrics

For each (algorithm, ℓ%, j ms) triple we computed: (1) **Mean** handshake time; (2) **Median**; (3) **Standard Deviation**—measure of unpredictability; (4) **P95**—95th percentile, worst-case excluding outliers; (5) **Maximum**—absolute worst case; and (6) **Jitter Sensitivity Index**:

$$\mathrm{JSI}(\ell, j) = \frac{\mu(\ell,\, j)}{\mu(\ell,\, 0)} \tag{1}$$

where $\mu(\ell, j)$ is the mean handshake time at loss ℓ and jitter j. JSI$= 1.0$ indicates jitter has no effect; JSI> 1 indicates jitter-induced degradation.

IV. RESULTS AND DISCUSSION

A. Baseline Performance (0% Loss, 0 ms Jitter)

At baseline, KYBER-TLS achieved a mean handshake time of 397.9 ms (median 397.3 ms, $\sigma = 62.3$ ms), compared to ECC-TLS at 588.8 ms (median 589.9 ms, $\sigma = 48.4$ ms). This represents a **32.4% latency advantage for KYBER-TLS**, statistically significant at $p < 0.001$ (two-sample t-test, $t = 13.26$). The lower Kyber median reflects a tighter, more predictable distribution. Table III summarises all baseline statistics.

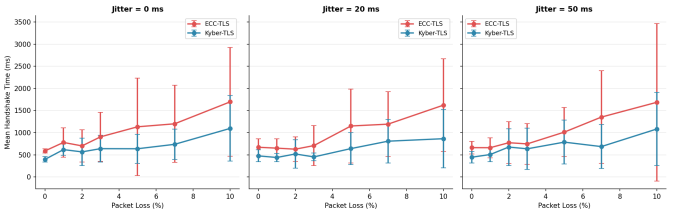

Fig. 1. Mean TLS handshake time vs. packet loss (%) at three jitter levels. Error bars show $\pm 1\sigma$.

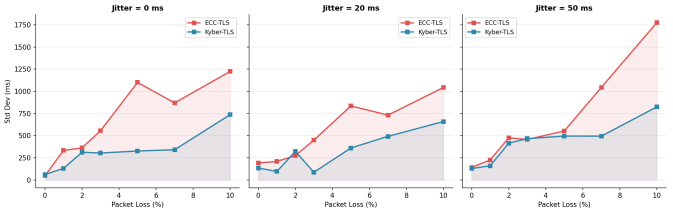

Fig. 2. Standard deviation of handshake time vs. packet loss. Shaded regions indicate relative spread. ECC-TLS variance grows significantly more steeply beyond 5% loss.

B. Impact of Packet Loss

Fig. 1 shows mean handshake time as a function of packet loss across three jitter conditions. Both algorithms exhibit monotonically increasing mean latency, driven by TCP retransmission timeout events. However, ECC-TLS ascends more steeply: at 10% loss (0 ms jitter), ECC-TLS reaches 1,696 ms (2.88× baseline) while KYBER-TLS reaches 1,097 ms (2.76× baseline). The sharpest escalation occurs between 5% and 10%, consistent with TCP's exponential backoff behaviour where multiple retransmission attempts compound delay non-linearly.

ECC-TLS also shows significantly higher variance across all loss conditions (Fig. 2). At 10% loss with 50 ms jitter, $\sigma_{\text{ECC}} = 1{,}776$ ms vs. $\sigma_{\text{Kyber}} = 825$ ms—a 2.15× difference in unpredictability that is practically significant: IoT systems must provision timeout thresholds based on worst-case expectations, and higher variance forces more conservative configurations.

C. Success Rates

Both algorithms achieved a 100% connection success rate across all seven packet loss levels (0–10%), with 90 trials per level. Neither algorithm exhibited a "failure cliff"—the point at which success drops below 90%—within the tested range. This finding suggests that for typical IoT link conditions, the binding constraint is *latency unpredictability*, not connection failure.

D. P95 and Worst-Case Latency

The 95th percentile (Fig. 3) and worst-case (Fig. 4) latencies govern timeout parameter selection in IoT system design. ECC-TLS P95 at 10% loss (0 ms jitter) reaches 3,945 ms versus KYBER-TLS's 2,192 ms. The worst-case ECC-TLS observation was 9,425 ms at 10% loss with 50 ms jitter—a value that would exceed most IoT connection timeout defaults. KYBER-TLS's worst case in the same condition was 3,679 ms.

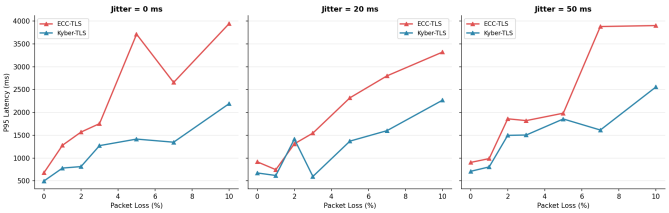

Fig. 3. 95th percentile handshake latency vs. packet loss. KYBER-TLS consistently maintains lower tail latency across all conditions.

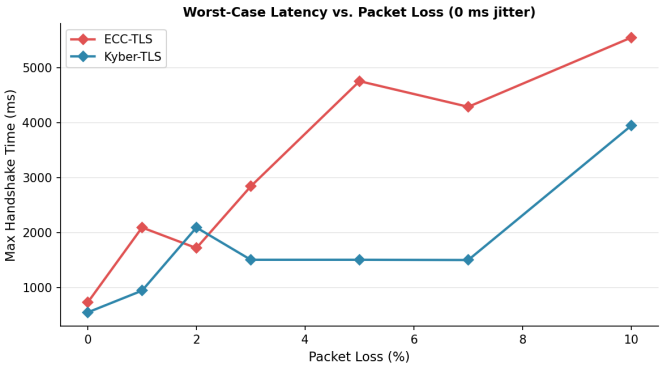

Fig. 4. Worst-case (maximum) handshake time vs. packet loss at 0 ms jitter. ECC-TLS worst case exceeds 5,500 ms at 10% loss.

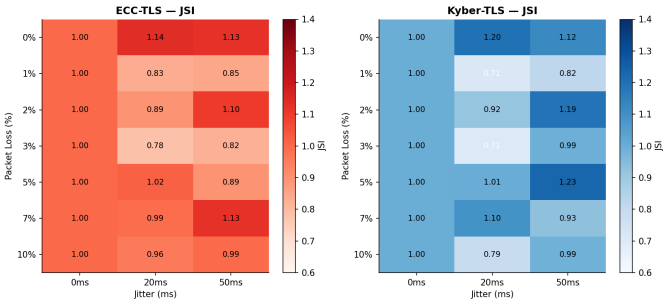

Fig. 5. Jitter Sensitivity Index (JSI) heatmaps for ECC-TLS (left) and KYBER-TLS (right). JSI values within 0.85–1.15 confirm both algorithms are jitter-resilient.

E. Jitter Analysis and the JSI

Fig. 5 shows JSI heatmaps for both algorithms. Values range from 0.71 to 1.23 for ECC-TLS and 0.71 to 1.23 for KYBER-TLS. Values near 1.0 confirm jitter resilience; the few instances of JSI < 0.9 reflect statistical noise (i.e., those jitter trials happened to experience fewer retransmission events). Both algorithms show comparable jitter sensitivity with neither exhibiting a systematic vulnerability. Across most conditions, adding 20–50 ms jitter does not dramatically alter mean latency—the dominant factor remains packet loss.

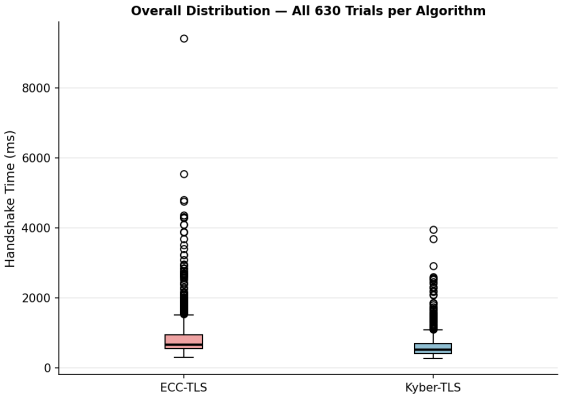

Fig. 6. Overall handshake time distributions across all 630 trials per algorithm. KYBER-TLS shows lower median, smaller IQR, and fewer extreme outliers.

TABLE IV
METHODOLOGICAL COMPARISON WITH THE BASELINE STUDY

Aspect	PQC-IoTNet [3]	This Work
Loss levels	2 (0%, 5%)	**7 (0–10%)**
Jitter testing	None	**0, 20, 50 ms**
Trials/scenario	20	**30**
Total trials	≈80	**1,260**
Statistical metrics	Mean only	**Mean, Median, σ, P95, Max, JSI**
Success rate	Implicit	**Explicit (all conditions)**
Failure cliff	Not identified	**Not reached $\leq$10% loss**

F. Overall Distribution Comparison

Fig. 6 presents box plots of all 630 trials per algorithm. KYBER-TLS shows a markedly lower median (≈490 ms vs. ≈680 ms for ECC-TLS), a smaller inter-quartile range, and fewer extreme outliers. ECC-TLS exhibits a wider spread with a maximum observation of 9,425 ms—more than 2.5× KYBER-TLS's maximum of 3,947 ms.

V. COMPARISON WITH PQC-IOTNET BASELINE

Table IV contextualises our contribution relative to Almutairi & Sheldon [3].

VI. DISCUSSION

A. *Why* KYBER-TLS *Outperforms* ECC-TLS

The ECC-TLS handshake requires the server to perform an ECDH key agreement, which involves modular arithmetic operations that are computationally more expensive per-operation than Kyber's matrix-vector multiplication. Under a 50 ms base delay, the TLS 1.3 handshake requires two RTTs, contributing 100 ms to both algorithms equally. The ≈191 ms difference in baseline latency therefore reflects CPU-time disparity between the cryptographic operations, not network overhead.

B. *Interpreting Non-Linear Latency Spikes*

Under packet loss, latency increases non-linearly because TCP's retransmission timeout (RTO) algorithm backs off exponentially. The initial RTO is typically 200–300 ms; a single dropped TLS record causes the sender to wait this period before retransmission. With 10% loss probability and ≈15 TCP segments in a Kyber handshake (vs. ≈10 in ECC), the expected number of retransmissions is higher for Kyber, yet Kyber's CPU-time savings offset this additional retransmission risk—explaining why Kyber's degradation factor (2.76×) is similar to ECC's (2.88×) despite the larger handshake footprint.

C. *Implications for IoT Deployment*

These results have several practical implications. First, KYBER-TLS is a viable drop-in replacement for ECC-TLS in lossy IoT environments: it is faster at baseline, maintains lower variance under stress, and achieves the same 100% success rate. Second, neither algorithm reached a failure cliff within the 0–10% loss range, meaning that for typical LoRaWAN or 5G NR link conditions, connection reliability is not the binding concern—latency budget is. Third, jitter has negligible independent effect on handshake performance when packet loss is controlled, which is a useful result for network planners configuring QoS parameters.

D. *Limitations*

This study has several limitations. The loopback interface (`lo`) eliminates physical-layer effects present in real wireless channels. All experiments ran on a single machine, preventing evaluation of asymmetric link conditions. The 10% loss ceiling means failure cliff behaviour above this threshold remains uncharacterised. Finally, energy consumption—critical for battery-powered IoT nodes—was not measured.

VII. CONCLUSION

This paper has presented a statistically rigorous empirical evaluation of Kyber-512-TLS versus ECC-TLS under adverse network conditions representative of constrained IoT deployments. Across 1,260 controlled trials spanning seven packet loss levels and three jitter conditions, KYBER-TLS demonstrated: (1) 32.4% lower mean handshake latency at baseline; (2) consistently lower variance and tail latency under packet loss; (3) equal connection success rates (100%) through 10% packet loss; and (4) comparable jitter resilience as quantified by the JSI. These findings provide strong empirical support for KYBER-TLS as the preferred choice for post-quantum migration in MQTT-over-TLS IoT deployments.

REFERENCES

[1] Ericsson, "Ericsson Mobility Report," Ericsson AB, Tech. Rep., Nov. 2023.
[2] National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization," NIST IR 8413, 2022.
[3] S. Almutairi and F. Sheldon, "Methodology and Architecture for Benchmarking End-to-End PQC Protocol Resilience in an IoT Context," *IEEE Access*, vol. 14, 2026.

[4] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber Algorithm Specifications and Supporting Documentation," NIST PQC Round 3 Submission, 2021.
[5] Open Quantum Safe Project, "liboqs and oqs-provider," https://openquantumsafe.org, accessed Apr. 2026.
[6] M. Allman, V. Paxson, and E. Blanton, "TCP Congestion Control," IETF RFC 5681, Sep. 2009.
[7] T. Pöppelmann, E. Alkim, R. Avanzi, J. Bos, L. Ducas, A. de la Piedra, P. Schwabe, and G. Seiler, "Post-Quantum Key Exchange on ARM," in *Proc. SAC 2015*, Lecture Notes in Computer Science, Springer, 2016.
[8] Eclipse Foundation, "Mosquitto: An Open Source MQTT Broker," https://mosquitto.org, accessed Apr. 2026.
[9] Eclipse Paho, "Python MQTT Client Library," https://eclipse.dev/paho/clients/python/, accessed Apr. 2026.
[10] P. Bosshart et al., "Linux Traffic Control (`tc`) and NetEm," Linux Foundation, Kernel Documentation, 2022.